

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA0307	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	20% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	Shaik Nuha	Reg. No.:	192472242
Section / Batch:	B.E-CSE-AI	Date:	

Code of Conduct

I Shaik Nuha(192472242) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sk.Nuha

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A language processing system must represent sentence structure for grammar checking and syntactic analysis. The developers are deciding between Context-Free Grammar (CFG) trees and dependency parsing structures. Analyze how these two approaches differ in representing sentence structure and justify which is more effective for capturing relationships between words.
2. A real-time application needs to parse user input efficiently, even when sentences are incomplete or ambiguous. The system currently uses top-down parsing, but an alternative is Earley parsing. Analyze the differences between these parsing techniques and reason which is more suitable for such dynamic input conditions.
3. An NLP model must handle ambiguity in sentences such as “She saw the man with a telescope.” The designers are considering CFG, PCFG, and neural parsing techniques. Analyze how these approaches differ in handling ambiguity and reason which method is most effective for real-world applications.
4. A grammar system needs to correctly handle subject–verb agreement and verb argument structures. The developers are comparing feature structures and subcategorization frames. Analyze how these approaches differ and justify which provides better support for enforcing grammatical correctness.
5. A large-scale NLP system must perform fast and accurate dependency parsing on big datasets. The choice is between transition-based parsing and graph-based parsing. Analyze the differences between these methods in terms of decision-making and performance, and justify which is more suitable for large-scale applications.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. CFG Trees vs Dependency Parsing

Analysis

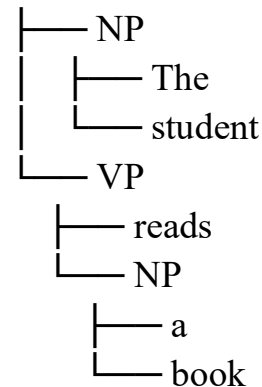
Both Context-Free Grammar (CFG) trees and dependency parsing represent sentence structure, but they focus on different aspects.

CFG / Constituency Parsing

CFG represents a sentence as a hierarchy of constituents or phrases.

Example: The student reads a book.

S



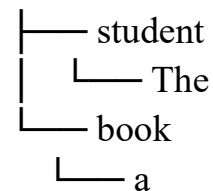
It shows how words form larger phrases such as:

- Noun Phrase (NP)
- Verb Phrase (VP)
- Prepositional Phrase (PP)
- Sentence (S)

Dependency Parsing

Dependency parsing directly represents relationships between individual words.

reads



Here, reads is the head, while student is its subject and book is its object.

Comparison

Aspect	CFG / Constituency	Dependency Parsing
Basic unit	Phrase/constituent	Word
Main focus	Phrase structure	Word-to-word relationships
Represents phrases	Excellent	Indirectly
Subject/object relationships	Indirect	Direct
Long-distance relationships	Less direct	Very useful
Semantic relation extraction	Less convenient	More convenient
Grammar analysis	Very useful	Useful
NLP information extraction	Moderate	Strong

Justification

For grammar checking and syntactic analysis, CFG trees are valuable because they clearly show phrase structure.

However, if the primary requirement is to capture relationships between words, dependency parsing is more effective.

For example:

The doctor prescribed medicine.

Dependency parsing directly captures:

prescribed → doctor (subject)

prescribed → medicine (object)

Conclusion

CFG is preferable for hierarchical phrase structure, while dependency parsing is preferable for word-to-word grammatical relationships. For modern NLP systems that need relation extraction, dependency parsing is generally more effective.

2. Top-Down Parsing vs Earley Parsing

Analysis

A real-time application may receive incomplete or ambiguous input such as:

"Book a flight..."

followed later by:

"to Delhi..."

and finally:

"with a window seat."

Top-Down Parsing

Top-down parsing begins with the start symbol and attempts to generate the input.

S

↓

VP

↓

Verb + NP

↓

Book + ...

It predicts structures before consuming all input.

Problems

1. Backtracking

If an incorrect rule is selected, the parser may need to return and try another.

2. Ambiguity

Multiple grammar rules may produce possible structures.

3. Incomplete input

It may not handle partial conversational input efficiently.

4. Real-time latency

Repeated prediction and backtracking can increase response time.

Earley Parsing

Earley parsing uses dynamic programming and maintains partial parsing states.

It has three major operations:

Predictor

Predicts possible grammar rules.

Scanner

Matches expected grammar symbols against input.

Completer

Completes partially recognized structures.

This allows the parser to maintain incomplete and alternative interpretations efficiently.

Comparison

Feature	Top-Down Parsing	Earley Parsing
Starting point	Grammar start symbol	Grammar + input positions
Backtracking	Can be high	Avoids repeated work using chart states
Ambiguity	Can be expensive	Maintains alternative parses
Partial input	Less suitable	Better suited
General CFG	Depends on implementation	Supports general CFG
Dynamic input	Less suitable	More suitable
Real-time applications	Limited for complex grammar	Better for complex/ambiguous input

Justification

For simple command grammars, top-down parsing can be fast and easy to implement.

However, for incomplete, ambiguous, dynamically arriving input, Earley parsing is more suitable because it maintains partial analyses and avoids repeatedly starting the parse from scratch.

Conclusion:

Earley parsing is the better choice for dynamic and ambiguous real-time input, particularly when the grammar is relatively general.

3. CFG vs PCFG vs Neural Parsing

Sentence

“She saw the man with a telescope.”

This sentence is ambiguous because “with a telescope” can have two different attachments.

Interpretation 1: She used the telescope

She

|

saw

/ \

she man

\

telescope

Meaning:

She used a telescope to see the man.

Interpretation 2: The man had the telescope

Meaning:

She saw a man who had a telescope.

The ambiguity is therefore primarily prepositional-phrase attachment ambiguity.

CFG

A CFG can generate both structures.

For example:

$VP \rightarrow VP PP$

$NP \rightarrow NP PP$

Both are grammatically possible.

Advantage

- Clear grammatical representation
- Simple
- Interpretable

Limitation

A basic CFG does not naturally determine which interpretation is more likely.

PCFG

A Probabilistic Context-Free Grammar assigns probabilities to grammar rules.

For example:

VP $\rightarrow$ VP PP 0.70

NP $\rightarrow$ NP PP 0.30

The parser can select the more probable parse based on learned probabilities.

Advantage

It provides a principled way to rank competing CFG parses.

Limitation

Its performance depends heavily on the quality and representativeness of the training data and grammar.

Neural Parsing

Neural parsers use machine-learning models to learn representations from large amounts of text.

They can use contextual information such as:

- Surrounding words
- Syntax
- Semantic relationships
- Lexical patterns
- Sentence context

For example, contextual information can help determine whether:

“with a telescope”

is more likely to describe the seeing action or the man.

Comparison

Feature	CFG	PCFG	Neural Parsing
Grammar rules	Explicit	Explicit + probabilities	Learned/implicit or hybrid
Ambiguity	Generates alternatives	Ranks alternatives	Uses contextual representations
Context sensitivity	Low	Moderate	High
Interpretability	High	High/Moderate	Lower
Training requirement	Low	Requires probabilistic estimates	Usually large datasets
Real-world robustness	Limited	Better	Generally strongest

Justification

For a simple controlled language, CFG may be sufficient.

For probabilistic ambiguity resolution, PCFG is better.

For real-world applications with varied language and contextual ambiguity, neural parsing is generally the most effective, particularly when trained on suitable domain data.

Conclusion:

A practical modern system can combine them:

CFG Grammar



Candidate Parses



Neural/Probabilistic Ranking



Best Interpretation

This provides both structural constraints and contextual disambiguation.

4. Feature Structures vs Subcategorization Frames**Analysis**

These two concepts solve related but different problems.

Feature Structures

Feature structures represent grammatical properties of linguistic units.

Typical features include:

Number = Singular

Person = Third

Tense = Present

Gender = ...

For example:

The student reads.

The subject:

Number = Singular

Person = Third

must agree with:

reads

Number = Singular

Person = Third

But:

The student read.

can also be grammatical depending on tense.

Feature structures therefore allow the grammar to enforce constraints between constituents.

Subcategorization Frames

A subcategorization frame specifies what types of arguments a verb expects.

For example:

Transitive verb

eat

eat → Subject + Verb + Object

Example:

The student eats food.

Ditransitive verb

give

give → Subject + Verb + Object + Recipient

Example:

The teacher gave the student a book.

Prepositional complement

depend

depend → Subject + Verb + on + NP

Example:

The result depends on the data.

Comparison

Aspect	Feature Structures	Subcategorization Frames
Main purpose	Represent grammatical features	Represent verb argument requirements
Agreement	Excellent	Not primary purpose
Number/person	Yes	No
Tense/features	Yes	No
Verb complements	Limited/directly not their focus	Excellent
Argument structure	Can encode features	Specifically describes it
Grammatical constraints	Strong	Strong for verb complements

Example

Consider: The doctors prescribe medicines.

Feature structures check:

doctors → Number = Plural

prescribe → Number = Plural

Subcategorization checks that:

prescribe → Subject + Object

Therefore: Doctors prescribe medicines.

satisfies both agreement and argument requirements.

Justification

If the requirement is specifically subject–verb agreement, feature structures provide better support.

If the requirement is verb argument structure, subcategorization frames are better.

Therefore, neither completely replaces the other.

Best solution

Use both:

Feature Structures

+

Subcategorization Frames

↓

Grammatical Validation

Conclusion:

Feature structures are better for enforcing agreement and grammatical feature compatibility, while subcategorization frames are better for enforcing verb argument structures. A grammar system should combine them for comprehensive grammatical correctness.

5. Transition-Based vs Graph-Based Dependency Parsing

Analysis

Large-scale NLP systems need dependency parsers that are both accurate and fast.

The two major approaches are transition-based parsing and graph-based parsing.

Transition-Based Parsing

Transition-based parsing constructs the dependency tree incrementally using a sequence of decisions.

Typical operations include:

SHIFT

REDUCE

LEFT-ARC

RIGHT-ARC

For example, the parser maintains:

Stack + Buffer

and repeatedly decides what transition should be performed.

Advantages

- Fast
- Often close to linear-time behaviour
- Low memory requirements
- Suitable for real-time applications

Limitation

The parser makes a sequence of local decisions. An early incorrect decision can affect later parsing decisions.

Graph-Based Parsing

Graph-based parsing treats dependency parsing as an optimization problem.

The parser:

1. Creates possible dependency arcs.
2. Assigns scores to them.
3. Searches for the highest-scoring valid dependency tree.

Conceptually:

Words



Possible Dependency Graph



Score Candidate Arcs



Find Best Tree



Dependency Parse

Advantages

- Considers global tree structure
- Can avoid some local decision errors
- Often provides strong parsing accuracy

Limitations

- Computationally more expensive
- More memory may be required
- Can be less suitable for strict low-latency applications

Comparison:

Feature	Transition-Based	Graph-Based
Decision style	Sequential/local	Global optimization
Speed	Generally faster	Generally slower
Memory	Lower	Higher
Error propagation	More susceptible	Less dependent on sequential decisions
Global structure	Limited	Strong
Real-time use	Excellent	Less suitable
Large-scale processing	Very suitable	Suitable with sufficient resources

Accuracy	Often strong	Often strong, especially with global modeling
----------	--------------	---

Industry Example

Suppose a company needs to process millions of documents.

If processing speed and resource efficiency are the highest priorities, transition-based parsing is attractive:

Large Dataset



Fast Sequential Parsing



Dependency Trees

If maximum structural accuracy is more important and computational resources are available, graph-based parsing can be preferable.

Justification

For large-scale applications, transition-based parsing is generally more suitable when the main requirements are:

- High throughput
- Low latency
- Lower memory usage
- Real-time processing

Graph-based parsing is preferable when:

- Global structural accuracy is critical
- Processing latency is less restrictive
- Sufficient computational resources are available

Conclusion:

For a large-scale industrial system where speed and scalability are major requirements, transition-based dependency parsing is generally the more practical choice. A hybrid system can also use a fast transition-based parser for normal processing and a more computationally intensive method for low-confidence cases.